

A Hyeron-Ready Python Architecture for the Chaos Language Algorithm

Ben Goertzel and Collaborators

July 3, 2026

Abstract

This document proposes an elegant Python library architecture for the Chaos Language Algorithm (CLA). The goal is to start with a simple, usable, pure-Python implementation, while arranging the abstractions so that later versions can move clustering, categorization, pattern mining, grammatical search, and scoring into Hyeron/Atomspace/MeTTa without a major rewrite. The central architectural idea is to separate the stable CLA domain model from replaceable reasoning backends. The stable domain model contains symbols, chunks, categories, rules, grammars, edits, proposals, parse objects, and MDL scores. The replaceable backends implement pattern mining, category induction, grammar search, scoring, persistence, and fact querying. A backend-neutral fact layer provides the migration path from ordinary Python dictionaries and counters to Atomspace-based AI algorithms.

Contents

1 Purpose and Audience	1
2 Background: CLA and the Key Distinction	1
3 Design Thesis	2
4 Recommended Package Name	2
5 Core Conceptual Model	3
6 Repository Layout	3
7 Core Data Types	5
8 Edits and Proposals	7
9 Primary Interfaces	8
10 Main Induction Loop	9
11 Default Pure-Python Implementation	9
11.1 MVP Components	9
11.2 Simple User API	10

12 Scoring Design	11
12.1 MDL Components	12
13 Search Design	12
14 The Hyperon-Ready Abstraction	13
14.1 Fact Objects	13
14.2 FactStore Protocol	14
14.3 Hyperon Adapter Sketch	14
15 Migration Path Toward Hyperon	15
16 User-Facing Model Object	15
16.1 Explanation Example	16
17 Example Pipeline Configurations	17
17.1 Configured Pure-Python Pipeline	17
17.2 Hyperon-Enhanced Pipeline	17
18 Practical Milestones	17
19 Testing Invariants	18
19.1 Chunk Invariants	18
19.2 Category Invariants	18
19.3 MDL Invariants	19
19.4 Reproducibility Invariants	19
19.5 Backend Invariants	19
20 Recommended pyproject.toml	19
21 Risks and Antipatterns	20
21.1 Antipattern: Hyperon Everywhere from Day One	20
21.2 Antipattern: Miners Mutate State	20
21.3 Antipattern: Categories as Long Chunks	20
21.4 Antipattern: Scoring Hidden in Search	20
22 Summary of the Architectural Move	20
A Appendix A: Coding-Agent Implementation Contract	21
B Appendix B: Minimal First Sprint	22
B.1 Sprint Inputs	22
B.2 Sprint Outputs	22
B.3 Sprint Tasks	22
C Appendix C: Second Sprint	22
C.1 Context Distribution	22
C.2 Category Proposal	23

D	Appendix D: Hyperon Integration Tasks	23
D.1	Task D1: Fact Projection	23
D.2	Task D2: MemoryFactStore	23
D.3	Task D3: HyperonFactStore	23
D.4	Task D4: HyperonPatternMiner	24
D.5	Task D5: HyperonCategoryInducer	24
E	Appendix E: Suggested Property Tests	24
F	Appendix F: Recommended Documentation Examples	24
F.1	Example 1: Repeated String	24
F.2	Example 2: Substitutable Contexts	25
F.3	Example 3: Logistic or Lorenz Symbolization	25
G	Appendix G: Final Instruction to Coding Agents	25

1 Purpose and Audience

This document has two audiences.

- (a) Human designers of the CLA software stack, who need a clear and durable decomposition of concepts.
- (b) Coding agents that will implement the first working library and later extend it toward Hyperon/Atomspace integration.

The guiding goal is not to make the first version maximally clever. The guiding goal is to make the first version simple, correct, useful, and hard to paint into a corner. The design should permit a migration path from a pure-Python symbolic grammar induction library to a Hyperon-backed cognitive grammar induction substrate.

2 Background: CLA and the Key Distinction

The Chaos Language Algorithm begins with a continuous or discrete time series, converts it into a symbolic string, and searches for a compact, predictive grammar explaining the resulting symbolic dynamics. The library design in this document assumes the corrected CLA distinction between two orthogonal forms of compression:

Chunking. A chunk or nonterminal abbreviates a recurring sequence. If a sequence β recurs often, one may introduce a production $N \rightarrow \beta$ and rewrite occurrences of β as N . This is sequential, syntagmatic, “and-then” compression.

Categorization. A meta-symbol or category abbreviates a set of mutually substitutable tokens. If tokens $v_1, \dots, v_k$ occur in sufficiently similar contexts, one may introduce a category $M = \{v_1, \dots, v_k\}$ and write rules using M as a slot. This is paradigmatic, “either-or” compression.

This distinction must be preserved in the code. It is the most important conceptual guardrail for the library.

Invariant 1 (Chunk-category separation). *A chunk token is defined by a production $N \rightarrow \beta$, where β is a sequence. A category token is defined by a set of substitutable members $M = \{v_1, \dots, v_k\}$. A category is not a compressed sequence, and a chunk is not a substitutability class.*

3 Design Thesis

The CLA library should have three layers:

```
+-----+
| User API: Pipeline, fit(), transform(), predict(), explain() |
+-----+
| Domain Core: Corpus, Token, Grammar, Category, Edit, Score |
+-----+
| Backends: memory, JSON, SQLite, NumPy, sklearn, Hyperon |
+-----+
```

The domain core is deliberately boring. It contains typed, deterministic Python data structures and pure transformations. The backend layer is where intelligence lives. In the first version, backends can use simple counters, n-gram scans, context vectors, greedy search, and approximate MDL scoring. Later, the same interfaces can be implemented using Hyperon Atomspaces, MeTTa pattern matching, PLN-like inference, metagraph rewrite search, or other cognitive methods.

Design Principle 1 (Backend neutrality). *The core CLA objects should not know how patterns were discovered. A chunk proposal may come from a Python n-gram counter, a SEQUITUR-like routine, an information-theoretic search, a Hyperon query, or a remote agent. The core induction loop should see all of these as typed **Proposal** objects.*

Design Principle 2 (Optional Hyperon). *Hyperon integration should be an optional adapter package, not a core runtime dependency. The first implementation should be installable and testable without native Hyperon bindings, DAS, MeTTa, or any external Atomspace service.*

4 Recommended Package Name

Avoid a generic package name such as `cla`, which can collide with other meanings. Better choices are:

```
chaoslang
chaosgram
clalib
chaos_language
```

The rest of this document uses `chaoslang`.

5 Core Conceptual Model

The library should expose a small number of central concepts.

Corpus. A symbolic sequence, or a collection of sequences, with optional metadata about the trajectory, source system, sampling interval, or partition.

Token. Something that can occur in a rewritten sequence. Tokens include base symbols, chunk tokens, and category tokens.

Base symbol. A token corresponding to an original cell in the symbolic partition.

Chunk or nonterminal. A token defined by a production $N \rightarrow \beta$, where β is a sequence of tokens.

Meta-symbol or category. A token-like class defined by a member set $M = \{v_1, \dots, v_k\}$. The members are tokens that can fill the same slot in a rule.

Production. A grammar rule, typically of the form $N \rightarrow \beta$, where β is a sequence of tokens and category slots.

Grammar state. The current vocabulary, rules, categories, rewritten corpus, score cache, parse information, and edit log.

Proposal. A possible change to the grammar state, such as adding a chunk, adding a category, merging categories, splitting a category, or pruning an unused rule.

Edit. A committed or commit-ready state transformation.

Score. A two-part description-length score,

$$L(G, S) = L(G) + L(S \mid G),$$

where $L(G)$ is the model cost and $L(S \mid G)$ is the cost of encoding the data given the grammar.

6 Repository Layout

A clean initial repository layout is as follows:

```
chaoslang/  
  __init__.py  
  
  core/  
    ids.py  
    tokens.py  
    corpus.py  
    grammar.py  
    categories.py  
    state.py  
    edits.py  
    proposals.py  
    scoring.py  
    facts.py  
    events.py  
  
  symbolic/  
    trajectory.py  
    embedding.py  
    partition.py
```

```
symbolizers.py

induction/
  engine.py
  loop.py
  acceptance.py
  search.py
  schedule.py

mining/
  protocols.py
  ngrams.py
  sequitur.py
  contexts.py
  distributional.py

categorization/
  protocols.py
  js_clusterer.py
  hard_categories.py
  soft_categories.py

scoring/
  protocols.py
  mdl.py
  codes.py
  priors.py

rewrite/
  matcher.py
  rewriter.py
  parser.py
  derivations.py

stores/
  protocols.py
  memory.py
  json_store.py
  sqlite_store.py

hyperon/
  __init__.py
  atoms.py
  store.py
  pattern_miner.py
  category_inducer.py
  grammar_search.py
  metta_templates.py
```

```

io/
  json.py
  yaml.py
  latex.py
  graphviz.py

examples/
  logistic_map.py
  lorenz.py
  symbolic_string.py

tests/
  unit/
  property/
  integration/
  hyperon_optional/

pyproject.toml
README.md
AGENTS.md

```

The dependency direction should be:

```

chaoslang.hyperon -> chaoslang.core
chaoslang.mining  -> chaoslang.core
chaoslang.scoring -> chaoslang.core
chaoslang.core    -> no backend-specific modules

```

Requirement 1 (Core dependency rule). *No module in `chaoslang.core` may import `chaoslang.hyperon`, `sklearn`, `scipy`, `networkx`, or any optional AI backend. The core must remain light, deterministic, and backend-neutral.*

7 Core Data Types

The first implementation should use immutable dataclasses and typed IDs. This avoids a heavy dependency stack and makes states easy to serialize, compare, replay, and test.

```

from __future__ import annotations

from dataclasses import dataclass, field
from typing import FrozenSet, Literal, Mapping, NewType

TokenId = NewType("TokenId", str)
RuleId = NewType("RuleId", str)
CategoryId = NewType("CategoryId", str)

```

```

@dataclass(frozen=True)
class Token:
    id: TokenId
    kind: Literal["base", "chunk", "category"]
    label: str
    level: int = 0
    attrs: Mapping[str, object] = field(default_factory=dict)

@dataclass(frozen=True)
class Production:
    id: RuleId
    lhs: TokenId
    rhs: tuple[TokenId, ...]
    weight: float = 1.0

@dataclass(frozen=True)
class Category:
    id: CategoryId
    token_id: TokenId
    members: FrozenSet[TokenId]
    weights: Mapping[TokenId, float] = field(default_factory=dict)

@dataclass(frozen=True)
class Corpus:
    sequences: tuple[tuple[TokenId, ...], ...]
    attrs: Mapping[str, object] = field(default_factory=dict)

@dataclass(frozen=True)
class Grammar:
    tokens: Mapping[TokenId, Token]
    productions: Mapping[RuleId, Production]
    categories: Mapping[CategoryId, Category]

@dataclass(frozen=True)
class Score:
    total_bits: float
    model_bits: float
    data_bits: float
    diagnostics: Mapping[str, float] = field(default_factory=dict)

@dataclass(frozen=True)
class GrammarState:

```



```

corpus: Corpus
grammar: Grammar
score: Score | None = None
edit_log: tuple["Edit", ...] = ()

```

The essential design feature is that categories are first-class objects. They are not encoded as productions with long right-hand sides. A category has members; a chunk has a production body. This distinction should be visible in the type system.

8 Edits and Proposals

The induction engine should not mutate a grammar directly. It should choose typed edits and apply them through a controlled edit applier. This provides reproducibility, rollback, distributed search, backend neutrality, and a clean path to Hyperon-backed search.

```

from dataclasses import dataclass, field
from typing import Mapping, Protocol

```

```

@dataclass(frozen=True)
class Edit:
    kind: str
    payload: Mapping[str, object]

```

```

@dataclass(frozen=True)
class Proposal:
    edit: Edit
    estimated_delta_bits: float | None = None
    provenance: Mapping[str, object] = field(default_factory=dict)

```

```

class EditApplier(Protocol):
    def apply(self, state: GrammarState, edit: Edit) -> GrammarState:
        ...

```

Useful edit kinds include:

```

AddBaseSymbol
AddChunkRule
RewriteOccurrences
AddCategory
MergeCategories
SplitCategory
AddCategorySensitiveRule
DeleteUnusedRule
PruneCategoryMember
RefinePartition
AcceptParse

```

Design Principle 3 (Proposals, not side effects). *Pattern miners, category inducers, and grammar searchers should return **Proposal** objects. They should not directly mutate the current **GrammarState**. State changes pass through **EditApplier** and are recorded in the edit log.*

9 Primary Interfaces

The main extension points should be Python Protocols rather than inheritance-heavy abstract base classes. This allows a plain Python object, a remote service proxy, or a Hyperon adapter to implement the same interface.

```
from typing import Iterable, Protocol
```

```
class PatternMiner(Protocol):
    """Find recurring sequential patterns: candidate chunks."""
    def propose_chunks(self, state: GrammarState) -> Iterable[Proposal]:
        ...

class CategoryInducer(Protocol):
    """Find substitutable token sets: candidate categories."""
    def propose_categories(self, state: GrammarState) -> Iterable[Proposal]:
        ...

class GrammarSearcher(Protocol):
    """Find broader grammar edits beyond simple chunk/category moves."""
    def propose_grammar_edits(self, state: GrammarState) -> Iterable[Proposal]:
        ...

class Scorer(Protocol):
    def score(self, state: GrammarState) -> Score:
        ...

    def delta(self, state: GrammarState, edit: Edit) -> float:
        ...

class AcceptancePolicy(Protocol):
    def accept(
        self,
        old_score: Score,
        new_score: Score,
        proposal: Proposal,
        step: int,
    ) -> bool:
        ...
```

```

class Store(Protocol):
    def save_state(self, state: GrammarState) -> str:
        ...

    def load_state(self, state_id: str) -> GrammarState:
        ...

    def record_proposal(self, proposal: Proposal) -> None:
        ...

```

10 Main Induction Loop

The main induction loop gathers proposals from several sources, scores the candidate states, and accepts changes according to a policy. It should not care whether the proposals came from Python counters, NumPy clustering, Hyperon pattern matching, or an LLM-assisted searcher.

Algorithm 1 CLA Library Top-Level Induction Loop

```

1: procedure FIT( $S_0, E, K$ )
2:    $G_0 \leftarrow$  initial grammar over symbols in  $S_0$ 
3:    $X_0 \leftarrow$  GrammarState( $S_0, G_0$ )
4:    $X_0.score \leftarrow$  E.scorer.score( $X_0$ )
5:   for  $t = 1$  to  $K$  do
6:      $P \leftarrow \emptyset$ 
7:     for all pattern miner  $m$  in E.pattern_miners do
8:        $P \leftarrow P \cup m.propose\_chunks(X_{t-1})$ 
9:     end for
10:    for all category inducer  $c$  in E.category_inducers do
11:       $P \leftarrow P \cup c.propose\_categories(X_{t-1})$ 
12:    end for
13:    for all grammar searcher  $q$  in E.grammar_searchers do
14:       $P \leftarrow P \cup q.propose\_grammar\_edits(X_{t-1})$ 
15:    end for
16:     $(X_t, accepted) \leftarrow$  SelectAndApplyBest( $X_{t-1}, P, E, t$ )
17:    if not accepted then
18:      break
19:    end if
20:  end for
21:  return  $X_t$ 
22: end procedure

```

11 Default Pure-Python Implementation

The first version should be intentionally simple. It should fit symbolic strings, discover repeated chunks, discover simple distributional categories, and improve a two-part MDL score.

Algorithm 2 Score, Select, and Apply a Proposal

```
1: procedure SELECTANDAPPLYBEST( $X, P, E, t$ )
2:    $old \leftarrow E.scorer.score(X)$ 
3:    $best\_state \leftarrow X$ 
4:    $best\_score \leftarrow old$ 
5:    $best\_proposal \leftarrow \text{None}$ 
6:   for all proposal  $p$  in  $P$  do
7:      $Y \leftarrow E.applier.apply(X, p.edit)$ 
8:      $new \leftarrow E.scorer.score(Y)$ 
9:     if  $E.acceptance.accept(old, new, p, t)$  then
10:      if  $new.total\_bits < best\_score.total\_bits$  then
11:         $best\_state \leftarrow Y$ 
12:         $best\_score \leftarrow new$ 
13:         $best\_proposal \leftarrow p$ 
14:      end if
15:    end if
16:  end for
17:  if  $best\_proposal$  is  $\text{None}$  then
18:    return  $(X, false)$ 
19:  else
20:     $Z \leftarrow \text{AppendEditLog}(best\_state, best\_proposal.edit, best\_score)$ 
21:    return  $(Z, true)$ 
22:  end if
23: end procedure
```

11.1 MVP Components

NGramPatternMiner. Counts repeated n-grams up to a maximum length and proposes **AddChunkRule** edits.

ContextDistributionInducer. Computes left and right context distributions for every token.

JensenShannonCategoryInducer. Clusters tokens whose context distributions are close under Jensen-Shannon divergence or another symmetric distributional distance.

TwoPartMDLScorer. Computes an approximate $L(G) + L(S | G)$ score.

GreedyAcceptancePolicy. Accepts strictly improving edits.

MemoryStore. Saves states in memory during a run.

JsonStore. Serializes grammars, categories, scores, and edit logs to disk.

11.2 Simple User API

A one-line symbolic-string API should work early:

```
from chaoslang import CLA

symbols = "a b c a b d a b c a b d".split()
```

```

cla = CLA.simple(
    max_ngram=6,
    category_threshold=0.15,
    max_steps=200,
)

model = cla.fit_symbols(symbols)

print(model.score)
print(model.grammar productions)
print(model.grammar.categories)
print(model.rewrite(symbols))

```

Continuous symbolic dynamics can be added through a symbolizer pipeline:

```

from chaoslang import CLA
from chaoslang.symbolic import DelayEmbedding, GridPartitioner

symbolizer = DelayEmbedding(dim=3, tau=8) >> GridPartitioner(bins=12)

cla = CLA.simple(symbolizer=symbolizer)
model = cla.fit_trajectory(time_series)

```

The symbolic-string path should come first. The trajectory-to-symbol path should be added after the grammar induction core is stable.

12 Scoring Design

The initial scorer can be approximate. It needs to be stable and useful, not theoretically perfect. Later, it can be replaced with a probabilistic grammar scorer, a Bayesian scorer, an information-bottleneck objective, or a Hyperon/Atomspace-backed inference process.

```

from dataclasses import dataclass

@dataclass(frozen=True)
class CodeLengthConfig:
    token_id_bits: str = "log_vocab"
    rule_overhead_bits: float = 1.0
    category_overhead_bits: float = 1.0
    member_overhead_bits: float = 1.0
    occurrence_bits: str = "empirical_entropy"

class TwoPartMDLScorer:
    def __init__(self, config: CodeLengthConfig = CodeLengthConfig()):
        self.config = config

```

```

def score(self, state: GrammarState) -> Score:
    model_bits = self._grammar_bits(state.grammar)
    data_bits = self._corpus_given_grammar_bits(
        state.corpus,
        state.grammar,
    )
    return Score(
        total_bits=model_bits + data_bits,
        model_bits=model_bits,
        data_bits=data_bits,
        diagnostics={
            "num_tokens": len(state.grammar.tokens),
            "num_rules": len(state.grammar productions),
            "num_categories": len(state.grammar.categories),
        },
    )

```

12.1 MDL Components

A practical first approximation can use:

$$\begin{aligned}
 L(G) &= L_{tok}(V) + L_{rule}(R) + L_{cat}(C), \\
 L(S \mid G) &= L_{parse}(S, G) + L_{resid}(S, G).
 \end{aligned}$$

Here V is the vocabulary, R is the set of productions, and C is the set of categories. The residual term may initially be zero if the grammar must reproduce the observed corpus exactly. Later it may encode exceptions, noisy emissions, or probabilistic alternatives.

13 Search Design

Search should be pluggable. Do not put too much search intelligence inside the pattern miners or category inducers. A miner proposes; the search layer decides.

Useful search policies include:

Greedy search. Try all proposals and accept the best strict improvement.

Beam search. Keep the top B grammar states at each depth.

Annealing search. Sometimes accept worse edits to escape local optima.

Hyperon-guided search. Ask Atomspace/MeTTa for promising edits, edit sequences, or abstractions.

Evolutionary search. Mutate grammars and select by score.

The public API should allow these policies to be swapped without changing state objects, scorers, or miners.

14 The Hyperon-Ready Abstraction

The key architectural move is to add a backend-neutral fact layer.

CLA core objects <-> backend-neutral Facts <-> Hyperon atoms

This is preferable to making all core CLA objects Hyperon atoms from day one. Directly making the core Hyperon-native too early would couple the entire package to a particular implementation substrate before the CLA abstractions are stable.

14.1 Fact Objects

Define a small fact representation:

```
from dataclasses import dataclass, field
from typing import Mapping

@dataclass(frozen=True)
class Fact:
    predicate: str
    args: tuple[object, ...]
    truth: float = 1.0
    attrs: Mapping[str, object] = field(default_factory=dict)
```

Every core object can be projected into facts:

```
Token(t1, base, "A")
Token(n7, chunk, "N7")
Production(r3, n7, [a, b, c])
Category(m2, [x, y, z])
Member(x, m2, weight=1.0)
Precedes(a, b, count=42)
LeftContext(x, a, count=17)
RightContext(x, b, count=19)
Proposal(p9, AddCategory, delta_bits=-12.7)
AcceptedEdit(e4, AddChunkRule)
```

14.2 FactStore Protocol

```
class FactStore(Protocol):
    def add(self, fact: Fact) -> None:
        ...

    def remove(self, fact: Fact) -> None:
        ...

    def query(self, pattern: FactPattern) -> Iterable[Fact]:
        ...
```

```
def transaction(self) -> "Transaction":
    ...
```

Then implement:

MemoryFactStore. A dictionary/set implementation for v0.1.

HyperonFactStore. An adapter that translates Fact objects to Atomspace atoms and query patterns to MeTTa/Atomspace queries.

14.3 Hyperon Adapter Sketch

All Hyperon-specific API calls should live in the adapter layer.

```
# chaoslang/hyperon/store.py

from chaoslang.core.facts import Fact, FactPattern

class HyperonFactStore:
    def __init__(self, space: object):
        self.space = space

    def add(self, fact: Fact) -> None:
        atom = fact_to_atom(fact)
        self._add_atom(atom)

    def query(self, pattern: FactPattern):
        atom_pattern = pattern_to_atom(pattern)
        for atom in self._query_atoms(atom_pattern):
            yield atom_to_fact(atom)

    def _add_atom(self, atom: object) -> None:
        # All Hyperon API calls live here.
        raise NotImplementedError

    def _query_atoms(self, atom_pattern: object):
        # All Hyperon pattern matching lives here.
        raise NotImplementedError
```

Hyperon-backed pattern mining should still return ordinary CLA proposals:

```
class HyperonPatternMiner:
    def __init__(self, store: HyperonFactStore, query_library):
        self.store = store
        self.query_library = query_library

    def propose_chunks(self, state: GrammarState):
```



```

self.store.save_state_facts(state)

pattern = self.query_library.repeated_sequence_pattern()
for fact in self.store.query(pattern):
    yield proposal_from_fact(fact)

```

Design Principle 4 (Hyperon returns proposals). *In the early and middle migration stages, Hyperon-backed components should return proposals, not directly mutated grammar states. The Python *InductionEngine* should still score, accept, apply, and log edits unless those components are intentionally replaced as well.*

15 Migration Path Toward Hyperon

The migration path should be incremental:

- v0.1.** Pure Python symbolic strings. `NGramPatternMiner` reads `Corpus` directly.
- v0.2.** Add fact projection. Core states can be projected into a `MemoryFactStore`.
- v0.3.** Add `HyperonFactStore`. It round-trips the same facts as `MemoryFactStore`.
- v0.4.** Add `HyperonPatternMiner`. It implements `PatternMiner` and returns chunk proposals.
- v0.5.** Add `HyperonCategoryInducer`. It implements `CategoryInducer` and returns category proposals.
- v0.6.** Add `HyperonGrammarSearcher`. It implements broader Atomspace-native grammar search, but still communicates through `Proposal` and `Edit` until the Hyperon backend is mature enough to own full search state.

At no stage should the public CLA API be forced to change.

16 User-Facing Model Object

The fitted model should expose both scientific and engineering views.

```

class CLAModel:
    @property
    def grammar(self) -> Grammar:
        ...

    @property
    def score(self) -> Score:
        ...

    @property
    def edit_log(self) -> tuple[Edit, ...]:
        ...

```

```

def rewrite(self, symbols: Sequence[str]) -> list[str]:
    ...

def parse(self, symbols: Sequence[str]) -> "ParseForest":
    ...

def predict_next(
    self,
    prefix: Sequence[str],
    k: int = 10,
) -> list[tuple[str, float]]:
    ...

def explain_token(self, token_id: TokenId) -> str:
    ...

def to_json(self) -> str:
    ...

def to_latex(self) -> str:
    ...

```

16.1 Explanation Example

The `explain_token` method should be built early. It will save substantial debugging effort and later provide a natural place for Atomspace-derived explanations.

Token M4 is a category:

```

members: A, D, F
induced from: similar left/right context distributions
accepted at step: 17
delta MDL: -23.4 bits
used in rules:
  N9 -> B M4 C

```

17 Example Pipeline Configurations

17.1 Configured Pure-Python Pipeline

```

from chaoslang import Pipeline
from chaoslang.mining import NGramPatternMiner
from chaoslang.categorization import JensenShannonCategoryInducer
from chaoslang.scoring import TwoPartMDLScorer
from chaoslang.induction import GreedyAcceptancePolicy

pipeline = Pipeline(
    pattern_miners=[NGramPatternMiner(max_n=8, min_count=2)],
    category_inducers=[JensenShannonCategoryInducer(threshold=0.12)],

```

```

        scorer=TwoPartMDLScorer(),
        acceptance=GreedyAcceptancePolicy(strict=True),
    )

model = pipeline.fit_symbols(symbols)

```

17.2 Hyperon-Enhanced Pipeline

```

from chaoslang import Pipeline
from chaoslang.hyperon import HyperonFactStore, HyperonPatternMiner
from chaoslang.categorization import JensenShannonCategoryInducer
from chaoslang.scoring import TwoPartMDLScorer
from chaoslang.induction import BeamSearchAcceptancePolicy

store = HyperonFactStore.from_default_space()

pipeline = Pipeline(
    pattern_miners=[
        HyperonPatternMiner(store=store),
    ],
    category_inducers=[
        JensenShannonCategoryInducer(threshold=0.12),
    ],
    scorer=TwoPartMDLScorer(),
    acceptance=BeamSearchAcceptancePolicy(width=16),
    store=store,
)

model = pipeline.fit_symbols(symbols)

```

The user-facing structure is the same. Only the backend components change.

18 Practical Milestones

A sane implementation order is:

1. **Domain core.** Implement IDs, tokens, categories, productions, grammar states, edits, proposals, and scores.
2. **Symbolic string MVP.** Fit from `list[str]`. Add `NGramPatternMiner`, simple chunk rewriting, approximate MDL, and a greedy induction loop.
3. **Categories.** Add context-distribution computation, hard category induction, category-sensitive rules, diagnostics, and token explanations.
4. **Persistence and reproducibility.** Add JSON grammar format, edit-log replay, deterministic random seeds, snapshot tests, and stable text renderings.
5. **Continuous symbolic dynamics.** Add delay embedding, grid partitioning, k-means partitioning, and a Symbolizer protocol.

6. **Fact layer.** Add `Fact`, `FactStore`, `MemoryFactStore`, and projection from grammar states to facts.
7. **Hyperon adapter.** Add `HyperonFactStore`, atom/fact conversion, and round-trip tests.
8. **Hyperon-native algorithms.** Add `HyperonPatternMiner`, `HyperonCategoryInducer`, and eventually `HyperonGrammarSearcher`.

19 Testing Invariants

The tests should enforce the theory, not just the current implementation.

19.1 Chunk Invariants

1. A chunk token must have exactly one production.
2. A production right-hand side must be non-empty.
3. Expanding a rewritten chunk must preserve the original corpus.
4. A production marked as useful must be used at least twice in the relevant corpus or grammar representation.

19.2 Category Invariants

1. A category token must have at least two members.
2. A category is not a production.
3. Category expansion denotes alternatives, not concatenation.
4. In hard-category mode, a token belongs to at most one category at a given level.
5. In soft-category mode, membership weights are explicit and normalized under the chosen convention.

19.3 MDL Invariants

1. Accepted greedy edits must not increase total description length.
2. `model_bits + data_bits == total_bits` up to numerical tolerance.
3. Removing unused rules should not worsen exact data reconstruction.
4. The same grammar and corpus should always receive the same score from a deterministic scorer.

19.4 Reproducibility Invariants

1. Same seed plus same input yields the same grammar.
2. Edit-log replay reconstructs the identical state.
3. JSON serialization round-trips all grammar objects and scores.

19.5 Backend Invariants

1. `MemoryFactStore` and `HyperonFactStore` must round-trip the same fact set.
2. Backend-specific miners must return valid `Proposal` objects.
3. A backend may rank proposals differently, but it must not violate the core type invariants.

20 Recommended `pyproject.toml`

```
[project]
name = "chaoslang"
version = "0.1.0"
description = "Chaos Language Algorithm grammar induction toolkit"
requires-python = ">=3.10"
dependencies = [
    "numpy>=1.26",
]

[project.optional-dependencies]
dev = [
    "pytest",
    "hypothesis",
    "ruff",
    "mypy",
]
symbolic = [
    "scipy",
    "scikit-learn",
]
hyperon = [
    "hyperon",
]
viz = [
    "graphviz",
]

[tool.ruff]
line-length = 100

[tool.mypy]
strict = true
```

Keep `hyperon` optional. The core package should install and run without native bindings or external services.

21 Risks and Antipatterns

21.1 Antipattern: Hyperon Everywhere from Day One

It is tempting to make every CLA object an Atomspace atom immediately. This risks coupling the entire library to backend APIs before the CLA domain model is stable. It also makes early testing and packaging harder. The safer path is to implement the domain model in pure Python and then project it into a fact layer.

21.2 Antipattern: Miners Mutate State

A miner that directly changes the grammar makes search hard to reason about. Miners should propose, not mutate. State changes should flow through `EditApplier` and be recorded in the edit log.

21.3 Antipattern: Categories as Long Chunks

Encoding $M = \{A, D, F\}$ as a production like $M \rightarrow ADF$ is wrong for CLA. The category M denotes a set of possible fillers, not the sequence A then D then F .

21.4 Antipattern: Scoring Hidden in Search

If a pattern miner secretly uses one scoring objective while the global search uses another, debugging becomes painful. Miners may estimate benefits, but accepted edits should be judged by the official scorer.

22 Summary of the Architectural Move

The library should begin as:

```
pure Python grammar state
pure Python miners
pure Python MDL scorer
pure Python edit applier
fact projection added early
Hyperon adapter added later
Hyperon-native cognition added last
```

The essential abstraction is:

CLA core objects $\leftrightarrow$ backend-neutral facts $\leftrightarrow$ Hyperon atoms.

This provides a practical tool quickly while leaving a credible path to deeper Hyperon-native intelligence: Atomspace-stored grammar graphs, MeTTa pattern mining over symbolic trajectories, category induction by graph and inference algorithms, and metagraph rewrite search over candidate grammars.

A Appendix A: Coding-Agent Implementation Contract

Place the following rules in AGENTS.md or equivalent project instructions.

Project rule 1:

Never represent a meta-symbol/category as a sequence-compressing rule.
Categories represent substitutable alternatives. Chunks represent sequences.

Project rule 2:

chaoslang.core must not import chaoslang.hyperon, sklearn, scipy, networkx, or optional backend packages.

Project rule 3:

All induction components return Proposal objects. Only EditApplier reconstructs or mutates GrammarState.

Project rule 4:

All accepted changes must be recorded in the edit log.

Project rule 5:

Every new backend must pass the same behavioral tests as MemoryFactStore.

Project rule 6:

Any stochastic algorithm must accept a seed.

Project rule 7:

Any grammar state must be serializable to JSON and reconstructable from JSON.

Project rule 8:

Hyperon integration belongs behind FactStore, PatternMiner, CategoryInducer, GrammarSearcher, or Scorer protocols.

B Appendix B: Minimal First Sprint

A first coding sprint should implement only the symbolic-string path. Do not implement continuous dynamics, Hyperon, or soft categories yet.

B.1 Sprint Inputs

Input: list[str]

Example: ["a", "b", "c", "a", "b", "d", "a", "b", "c"]

B.2 Sprint Outputs

1. A CLAModel containing a grammar, score, and edit log.

2. A JSON representation of the grammar.
3. A rewritten sequence using accepted chunk tokens.
4. A human-readable explanation of each chunk and category.

B.3 Sprint Tasks

1. Implement `Token`, `Production`, `Category`, `Corpus`, `Grammar`, `Score`, and `GrammarState`.
2. Implement `Edit`, `Proposal`, and `EditApplier`.
3. Implement `NGramPatternMiner`.
4. Implement `TwoPartMDLScorer` with a simple approximate code length.
5. Implement greedy proposal selection.
6. Implement JSON save/load.
7. Implement tests for chunk-category separation.

C Appendix C: Second Sprint

The second sprint should add hard categories.

C.1 Context Distribution

For a token v , compute:

$$P_L(a \mid v) = \frac{\#(a, v)}{\sum_x \#(x, v)},$$

$$P_R(b \mid v) = \frac{\#(v, b)}{\sum_y \#(v, y)}.$$

Then define the context vector:

$$c(v) = (P_L(\cdot \mid v), P_R(\cdot \mid v)).$$

Tokens with sufficiently similar context vectors become candidate category members.

C.2 Category Proposal

D Appendix D: Hyperon Integration Tasks

Hyperon integration should be added only after the pure-Python fact layer is tested.

Algorithm 3 Simple Hard Category Proposal

```
1: procedure PROPOSECATEGORIES( $X, \theta, min\_members$ )
2:    $C \leftarrow \text{ComputeContextVectors}(X.corpus)$ 
3:    $clusters \leftarrow \text{ClusterByDistance}(C, \theta)$ 
4:    $P \leftarrow \emptyset$ 
5:   for all cluster  $K$  in  $clusters$  do
6:     if  $|K| \geq min\_members$  then
7:        $e \leftarrow \text{Edit}(\text{"AddCategory"}, K)$ 
8:        $P \leftarrow P \cup \{Proposal(e)\}$ 
9:     end if
10:  end for
11:  return  $P$ 
12: end procedure
```

D.1 Task D1: Fact Projection

Implement:

```
def state_to_facts(state: GrammarState) -> Iterable[Fact]:
    ...
```

```
def facts_to_state(facts: Iterable[Fact]) -> GrammarState:
    ...
```

The round trip should preserve all token IDs, rule IDs, category IDs, productions, member sets, weights, and corpus sequences.

D.2 Task D2: MemoryFactStore

Implement a simple in-memory fact store with pattern matching over predicate name and argument wildcards. This is not meant to be clever; it is a reference backend for tests.

D.3 Task D3: HyperonFactStore

Implement conversion functions:

```
def fact_to_atom(fact: Fact) -> object:
    ...
```

```
def atom_to_fact(atom: object) -> Fact:
    ...
```

```
def pattern_to_atom(pattern: FactPattern) -> object:
    ...
```

All direct Hyperon calls should stay inside this module.

D.4 Task D4: HyperonPatternMiner

Implement a minimal repeated-sequence query that returns the same sort of `Proposal` as `NGramPatternMiner`. The first goal is behavioral equivalence on tiny test corpora, not performance.

D.5 Task D5: HyperonCategoryInducer

Represent context facts in the Atomspace and implement a simple category proposal query. Again, the first target is equivalence with the pure Python inducer on tiny examples.

E Appendix E: Suggested Property Tests

Use property-based tests where possible.

1. Generate random token sequences. Add a chunk. Expand the chunk. Verify the original sequence is recovered.
2. Generate random categories. Verify category expansion denotes a set of alternatives, not a concatenation of members.
3. Generate small grammars and serialize to JSON. Deserialize and compare canonical forms.
4. Generate a grammar state and project to facts. Reconstruct from facts. Compare canonical forms.
5. Run the same stochastic pipeline twice with the same seed. Verify identical edit logs.
6. Compare `MemoryFactStore` and `HyperonFactStore` on the same fact queries once the Hyperon adapter exists.

F Appendix F: Recommended Documentation Examples

The README should include three runnable examples.

F.1 Example 1: Repeated String

A toy string with obvious repeated chunks, used to explain productions.

F.2 Example 2: Substitutable Contexts

A toy string such as:

```
a x b a y b a z b c x d c y d c z d
```

This should lead to a category roughly like:

$M1 = \{x, y, z\}$

because x , y , and z occur in similar contexts.

F.3 Example 3: Logistic or Lorenz Symbolization

A small continuous trajectory example can be added later, after the symbolic core is stable.

G Appendix G: Final Instruction to Coding Agents

Implement chaoslang as a pure-Python CLA grammar induction library first. Keep the core domain model small and immutable. Separate chunking from categorization at the type level. Make every induction component return Proposal objects. Route all state changes through EditApplier. Use a simple two-part MDL scorer and greedy search initially. Add a backend-neutral Fact layer early, but do not depend on Hyperon in the core. After the pure-Python system passes tests, add HyperonFactStore and then Hyperon-backed PatternMiner, CategoryInducer, and GrammarSearcher implementations behind the same protocols. Never change the public API just because a backend changes.